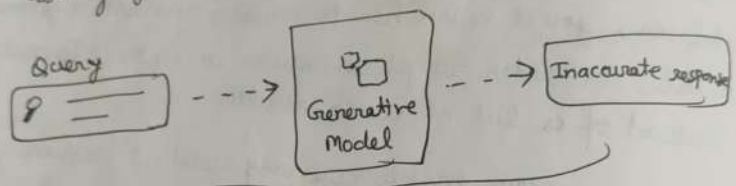
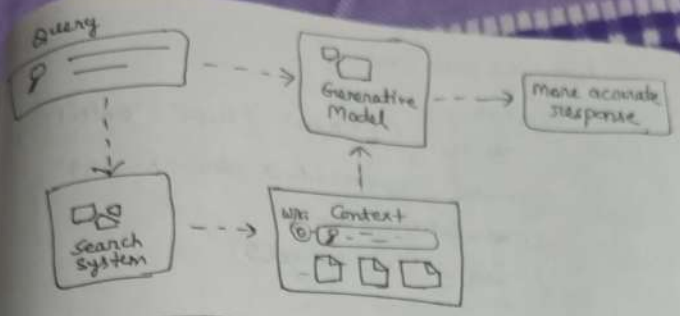
But if I go & search a few chapters in books about thermodynamics. I give them to my friend, and then I ask them to answer the question based on that, they are much more likely to answer the question correctly.

In this chapter we'll compare 2 ways a large language model can answer a question. The first one is by feeding the question to the generative model, and obtaining the answer. This will generate an answer that could be correct, but it may not. This is equivalent to asking your friend a thermodynamics question.



A generative model receives a query & outputs a response. This response may be inaccurate.

↳ The second one starts by using a search system to retrieve documents where the answer to the query is likely to appear. Then we feed the question and the documents to a generative model, and prompt it to answer the question using these documents. This yields a more accurate response.



The query is first given to a search system, which retrieves documents which are likely to contain the answer. Then the query and the documents (context) are fed to the generative model, for a more accurate response.

Generating Answers (Without Search)

first use a generative model to answer a slightly question — w/o search. We are trying to find out how people won more than one Nobel prize. So, we ask the the following query.

"How many people have won more than one Nobel prize?"

answer to this question is five: Marie Curie, Linus Pauling, John Bardeen, Frederick Sanger, and Karl Barry Sharpless.

way to ask this to the model is with the following of code, which calls the co.chat endpoint.

```
prediction_without_search = [
    co.chat(
        messages=[{"role": "user", "content": query}],
        model="command-r-plus-08-2024",
        max_tokens=50,
    ) for _ in range(5)
]
```

We call the endpoint five times to get five responses. The max_tokens parameter determines the length of the answer (which is why some answers appear truncated).

Responses:

- Marie Sklodowska-Curie, a Polish and naturalized French physicist . . .
- . . . .
- . . .
- . . .
- . . . ,

These answers sound like they could be correct, but they're all wrong. One reason for this is that transformers are good at talking and understanding sentiment and nuisances of the language etc, but not so good at storing info^n. As a matter of fact, storing info^n inside the nodes of the NN is not something that we can (or should) fully trust. Instead, let's first search for the answer using what we've learned in the prev. sections of this post.

Searching Answers

In order to find the answer to this question in the Wikipedia dataset (the one we've been working with throughout this post), we can use the same dense_retrieval function that we used before. For simplicity, we will only use dense retrieval without Rerank, but we invite you to add it to the lab & see how the results improve!

```
responses = dense_retrieval(query, num_results=20)
```

This retrieves the top 20 articles, with their corresponding paragraphs. Here are the top 3 (remember that the search is done by finding the most similar paragraphs to the query, so same articles may appear several times with different paragraphs).

Responses:

- Nobel Peace Prize: ", the Peace Prize has been - - -
- Nobel Prize: "The strict rule - - - ..."
- Nobel Prize: "The prize - - - ".

Next, we'll feed these 20 paragraphs to a generative model, and instruct it to answer the question in sentence format.

↳ Generating an Answer from the Search Results

In order to get the generative model to answer a question based on a certain context, we need to create a prompt. And in this prompt, we need to give it a command and a context. The context will be the concatenation of all the paragraphs retrieved in the search step, which we can obtain using this line of code:

```
context = [r['text'] for r in responses]
```

The array "context" contains a lot of text, and, given the good results we've been obtaining with search mechanisms, we are fairly confident that somewhere in this text lies the answer to our original question. Now, we invoke the "Chat" endpoint. The prompt we'll use is the following.

```
prompt = f"""
Use the Information provided below to answer the questions
at the end. If the answer to the question is not
contained in the provided Info^n, say "The answer
is not in the context".

---
Context info^n:
{context}
---
Question: How many people have won more than one
Nobel prize?
"""
```

In other words, we've prompted the model to answer the question, but only from info coming from the "context" array. And if the info is not there, we are prompting the model to state that the answer is not in the context. The following line of code will run the prompt. As before, we generate 5 answers, and max_tokens controls the length of each answer.

```
prediction_with_search = [
        co.chat(
            messages = [{"role": "user", "content": prompt}
                        ],
            model = "command-r-plus-08-2024",
            max_tokens=50,
        ) for _ in range(5)
    ]
```

The 5 responses we get are the following (just like before, they are truncated):

- Five people have won multiple Nobel prizes - -
- - -
- - -
- - -
- - -

As you can see, this improved the quality of the answer. It got the right no. of people who received more than one Nobel prize, which is 5.

⇒ Conclusion

Generative models are prone to hallucinations. For ex., when asked a question, they may answer with an incorrect answer. In this chapter, you learned to power a generative model with search, in order to generate more accurate answers & reduce the chance of hallucinations.

Prompt Engineering

⇒ Constructing Prompts

When working with large language models (LLMs), the prompt is the key to getting the desired response. A well-designed prompt will result in useful & accurate responses from a model & will considerably improve your experience interacting with it.

Prompts can be as simple as one-liner or they can be as complex as multiple layers of specific info^n. The more specific your command is, the more likely you will get exactly what you need from the model.

↳ Setup.

```
! pip Install cohere
import cohere
co = cohere.ClientV2 ("Cohere API Key").
```

Let's also define a func^n to take a prompt & a temperature value & then call the Chat endpoint, which is how we can access the Command model. Here, we select the model type to be command. We set a default temperature value of 0, which nudges the response to be more predictable & less random. Throughout this article, you'll see diff. temperature values being used in diff. situations. Increasing the temperature value tells the model to generate less predictable responses & instead be more "creative".

This function pri

def generate_t

response =

messa

mode

temp

for event

if e

↳ Instruction

Pro

Write

Explain

While prompts

it does not

prompting

to a text

Hence,

Is a sha

the prompt
well-designed
uses from a
ce interacting

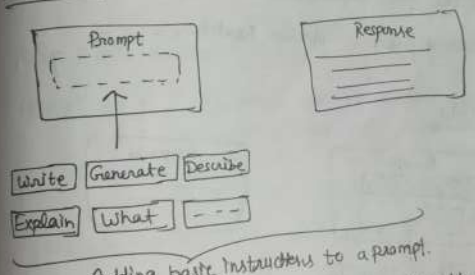
an be as
more specific
exactly what

erature value
can access
be command
the response
this article,
situations.
generate
ative".

function prints the text response generated by the mode

```
def generate_text(prompt, temp=0):
    response = co.chat_stream(
        message = prompt,
        model = "command-r-plus-08-2024",
        temperature = temp)
    for event in response:
        if event.event_type == "text-generation":
            print(event.text, end = '')
```

Instruction



Adding basic instructions to a prompt.

While prompts can morph into something very lengthy & complex, it does not have to be that way all the time. At its core, prompting a command model is about sending an instruction to a text generation model and getting a response back. Hence, the smallest unit of a perfectly complete prompt is a short line of instruction to the model.

Let's say we want to generate a product description for a wireless headphone. Here's an example prompt, where we create a variable for the user to input some text & merge that into the main prompt.

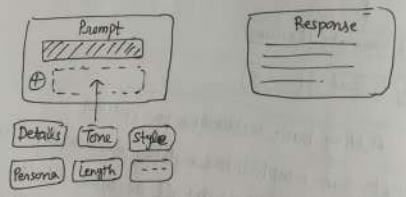
```
user_input = "a wireless headphone product named the CO-1T"
prompt = f"""Write a creative product description
            for {user_input}"""

generate_text (prompt, temp = 0.5)
```

The model returns the following sample response, which does the job we asked for:

[Looking for a wireless headphone - - -
- - - -]

↳ Specifics



Adding specific details to a prompt.

uct description for a
ompt, where we create
e text & merge that

uct named the CO-IT
product description
" " "

response, which

]

t.

mple & short prompt can get you started, but in most cases you'll
to add specificity to your instructions. A generic prompt will
turn a generic response & in most cases, that's not what we want.
the same way that specific instructions will help humans do our
k well, a model needs to be supplied with specific details to guide
is response.

ing back to the previ. prompt, the generated product description
s great, but what if we wanted it to include specific things
s its features, who it is designed for & so on? We can
struct the prompt to take more inputs from the user, like so:

```
user_input_product = "a wireless headphone product named the co-IT"
user_input_keywords = "bluetooth", "wireless", "fast charging"
user_input_customer = " a software developer who works in
                        noisy offices"
user_input_describe = "benefits of this product"

prompt = f """ Write a creative product description for
               {user_input_product}, with the keywords {user_input
               keywords} for {user_input_customer}, and
               describe {user_input_describe}. """

generate_text(prompt, temp=0.5)
```

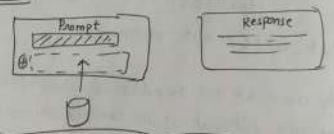
In the example above, we pack the additional details of the prompt in a single paragraph. Alternatively, we can also compose it to be more structured. ~~like so:~~ And here's an example response. This time, the product description is tailored more specifically to our desired target customer, includes the key features that we specified & sprinkles benefit statements throughout.

[Are you a software developer struggling to focus in hustle & bustle
— — —

There are many angles to add specificity to a prompt. Here are some examples:

- Style: Telling the model to provide a response that follows a certain style or framework. For ex., Instead of asking the model to "Generate an ad copy for a wireless headphone product" in generic sense, we ask it to follow a certain style, such as "Generate an ad copy for a wireless headphone product, following the AIDA framework – Attention, Interest, Desire, Action."
- Tone: Adding a line mentioning how the tone of a piece of text should be, such as professional, inspirational, fun, serious & so on. For ex., "Tone: casual".
- Persona: Telling the model to act like a certain persona helps to add originality & quality to the response. For ex., "You are a world-class content marketer. Write a product description for..."
- Length: Telling the model to generate text with a specific length, be it in words, paragraphs & others. This helps the model to be verbose, concise or somewhere in b/w. For ex., "Write in 3 paragraphs the benefits of..."

↳ Context.



Adding contextual info to a prompt

While LLMs excel in text generation tasks, they struggle in context-aware scenarios. Here's an example. If you were to ask the model for the top qualities to look for in wireless headphones, it will duly generate a solid list of points.

But if you were to ask
CO-IT headphone. It will
response because it does

In real app's, being able
because this is what
for a team or company
such as intelligent a
productivity tools, t
a wide range of

Here's an example wh
features of the CO-
context:

user_input = "What a
prompt = user_input
generate_text(p

This generates a
since it doesn't h

"The CO-IT wir

And here's the sam
the product descript

context = """

user_input =
prompt = f"
Given t
{user_i
generate_te